



RAAM: A Restricted Adversarial Attack Model with Adding Perturbations to Traffic Features

Peishuai Sun, Shuhao Li, Jiang Xie, Hongbo Xu, Zhenyu Cheng, Rui Qin

► To cite this version:

Peishuai Sun, Shuhao Li, Jiang Xie, Hongbo Xu, Zhenyu Cheng, et al.. RAAM: A Restricted Adversarial Attack Model with Adding Perturbations to Traffic Features. 37th IFIP International Conference on ICT Systems Security and Privacy Protection (SEC), Jun 2022, Copenhagen, Denmark. pp.126-141, 10.1007/978-3-031-06975-8_8 . hal-05319918

HAL Id: hal-05319918

<https://inria.hal.science/hal-05319918v1>

Submitted on 17 Oct 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons Attribution 4.0 International License



This document is the original author manuscript of a paper submitted to an IFIP conference proceedings or other IFIP publication by Springer Nature. As such, there may be some differences in the official published version of the paper. Such differences, if any, are usually due to reformatting during preparation for publication or minor corrections made by the author(s) during final proofreading of the publication manuscript.



RAAM: A Restricted Adversarial Attack Model with Adding Perturbations to Traffic Features

Peishuai Sun^{1,2}, Shuhao Li^{1,2}, Jiang Xie^{1,2}, Hongbo Xu^{1,2},
Zhenyu Cheng^{1,2}, and Rui Qin^{1,2}

¹ Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
{sunpeishuai,lishuhao,xiejiang,xuhongbo,chengzhenyu,qinrui}@iie.ac.cn

² School of Cyber Security, University of Chinese Academy of Sciences,
Beijing, China

Abstract. In recent years, intrusion detection system (IDS) based on machine learning (ML) algorithms has developed rapidly. However, ML algorithms are easily attacked by adversarial examples, and many attackers add perturbations to features of malicious traffic to escape ML-based IDSs. Unfortunately, most attack methods add perturbations without sufficient restrictions, generating unpractical adversarial examples. In this paper, we propose *RAAM*, a restricted adversarial attack model with adding perturbations to traffic features, which escapes ML-based IDSs. *RAAM* employs the improved loss to enhance the adversarial effect uses regularizer and masking vectors to restrict perturbations. Compared with previous work, *RAAM* can generate adversarial examples with superior characteristics: regularization, maliciousness and small perturbation. We conduct experiments on the well-known NSL-KDD dataset, and test on nine different ML-based IDSs. Experimental results show that the mean evasion increase rate (*EIR*) of *RAAM* is 94.1% in multiple attacks, which is 9.2% higher than the best of related methods, *DIGFuPAS*. Especially, adversarial examples generated by *RAAM* have lower perturbations, and the mean distance of perturbations (L_2) is 1.79, which is 0.81 lower than *DIGFuPAS*. In addition, we retrain IDSs with adversarial examples to improve their robustness. Experimental results show that retrained IDSs not only maintain the ability of detection for original examples, but also are hard to be attacked again.

Keywords: Adversarial attack · Machine learning · Intrusion detection system

1 Introduction

An Intrusion Detection System (IDS) is typically placed in a network to monitor traffic and detect malicious activity. It classifies traffic into different categories based on features, and reports malicious results to the network administrator. Traditional IDSs use rules to detect malicious traffic, but their efficiency

and generalization cannot adapt to constantly changing attack patterns. Fortunately, with the development of artificial intelligence, many machine learning (ML) algorithms have been proposed for IDSs. Usually, features of network traffic are extracted, and models are trained offline based on ML algorithms. Then the trained models are deployed on IDSs for online detection. Researches show that ML-based IDSs have stronger generalization and accuracy.

Although ML algorithms are widely used in various fields, some researchers find that ML algorithms have low robustness. In computer vision, Szegedy *et al.* [1] firstly propose the concept of adversarial examples. They find that only small perturbations on original images can mislead the ML system to wrongly classify images that humans easily classify correctly. Since then, lots of adversarial attack methods have been proposed in various fields [13, 14, 16–18]. However, these methods are not applicable for network traffic analysis. The traffic is composed of a discrete bytes stream. Compared with images, traffic is not differentiable. In addition, network protocols have very complex rules, and minor traffic changes can easily cause a protocol session to fail. Therefore, researchers focus on features of traffic, and generate adversarial features rather than actual traffic.

Generative Adversarial Network (GAN) [20] is trained through the adversarial game of the generator and the discriminator to generate more realistic and credible examples. The goal of the generator is to learn generating examples that can escape the detection of the discriminator. On the contrary, the discriminator is to distinguish between examples from original and generated data. Many researchers apply it to the generation of adversarial examples due to the strong ability of generation. Lin *et al.* [4] first use GAN to generate adversarial features of traffic for attacking IDSs, and get a high evasion increase rate (EIR). However, like most adversarial attacks methods, they do not restrict attacks insufficiently. This can make adversarial examples ineffective and unpractical.

In the paper, we employ GAN to generate perturbations. Different from previous work, we do not directly add perturbations to traffic features as adversarial examples. In particular, we have strict restrictions on adversarial attacks to ensure the authenticity of adversarial examples. Our main contributions are as follows:

- We propose a restricted adversarial attack model called *RAAM*. It mainly includes the following parts: (1) WGAN [19] for generating perturbations, (2) masking vectors for keeping the functional features unchanged, (3) regularizer R for controlling the distance of perturbations. Experimental results show that adversarial examples generated by *RAAM* have small perturbations due to the constraints it imposes on attack traffic.
- We employ the improved loss for WGAN. In particular, it does not simply imitate features of benign traffic, but search blind spots and boundary of ML-based IDSs. Experimental results show that generated adversarial examples by *RAAM* can attack ML-based IDSs better than previous methods.
- We design a prototype system based on *RAAM*, and conduct adversarial attack experiments on the well-known NSL-KDD dataset [21]. *RAAM* attacks

nine ML-based IDSs (MLP, RNN, SVM *et al.*), and the mean *EIR* is 94.10%, which is 9.2% higher than the advanced attack method, *DIGFuPAS* [6]. The mean L_2 is 1.79, which is 0.81 lower than *DIGFuPAS*. In addition, we use *RAAM* to enhance the robustness of IDSs. Simply put, *RAAM* generates adversarial examples to retrain IDSs for defending adversarial attacks. Experiments show that the retrained IDSs not only maintain the ability to detect the original examples but also are hard to be attacked.

The remainder of this paper is organized as follows. Section 2 introduces the related work. In Sect. 3, we analyze scene and motivation of adversarial attacks. Methodology is proposed in Sect. 4. Subsequently, in Sect. 5, we evaluate our method and show the relevant experimental results. Finally, we discuss and summarize in Sect. 6 and Sect. 7, respectively.

2 Related Work

In the field of network traffic, IDSs are always a hot issue because of the complexity and variability of attack and defense adversary. In recent years, IDSs based on ML algorithms emerge gradually [9–12]. Their performance is recognized by academia and industry. However, some researchers find that ML algorithms have low robustness and are easily confused by adversarial attacks. Based on different scenes, we divide adversarial attacks into white-box and black-box.

2.1 White-Box Adversarial Attack

A white-box adversarial attack means that attackers know all the parameters and structures of ML models, and can attack pointedly. Aiken and Scott-Hayward [2] propose a method to escape the detection of ML algorithms in distributed denial of service (DDoS). They select three features to add perturbations, and successfully reduce the detection rate of IDSs from 100% to 0%. However, their method only focuses on DDoS attacks without discussing other classes of attacks, and the selected features are relatively single.

Huang *et al.* [3] conduct adversarial experiments on three deep learning (DL) algorithms. They mainly use three measures: fast gradient sign method (FGSM), jacobian-based saliency map attack (JSMA), and JSMA reverse (JSMA-RE) [14, 15]. The generated scanning traffic is highly adversarial to IDSs. However, they do not restrict adversarial examples, which means that the generated examples are illegal.

Hashemi *et al.* [8] generate adversarial features and then transform them into actual traffic space. It is the first time that they legally modify network traffic to escape IDSs.

However, it is difficult to realize in practice, since attackers cannot completely know the information of model. Corresponding to the white-box, the black-box attack is more realistic.

2.2 Black-Box Adversarial Attack

A black-box adversarial attack means that attackers know nothing about structures of ML model, including training parameters, and defense methods, *etc.* Attackers can only interact with the target model through output results. Han *et al.* [7] generate practical adversarial traffic in the black-box scene. They consider the cost of attacks and use adversarial machine learning techniques to search for features located at the decision boundary of the ML model. However, they only use GAN to select features instead of generating, and the adversarial performance is relatively weak.

Recently, GAN is a popular method for adversarial attacks. Lin *et al.* [4] are the first to propose an adversarial model based on GAN to attack IDSs, called *IDSGAN*. *IDSGAN* is divided into three parts: an IDS as the attacked black-box model, a discriminator as a classification model to learn the black-box model, and a generator as the generation model to generate adversarial malicious examples. Experiments show that adversarial examples generated by *IDSGAN* can significantly reduce the detection rate of IDSs.

However, Usama *et al.* [5] point out that *IDSGAN* modifies two functional features of network traffic, which makes the generated traffic invalid. They make improvements on the basis of *IDSGAN* to ensure that all modified features are nonfunctional, and functional features remain the same. However, the original GAN used by Usama *et al.* has no obvious effect on IDSs, and the detection rate of IDSs does not drop significantly.

Duy *et al.* [6] propose an improved framework based on WGAN, called *DIGFuPAS*. They also modify the nonfunctional features for malicious traffic, and use three different inputs to compare the performance of generating examples. In addition, they conduct experiments on multiple datasets to prove the versatility and generalization of the *DIGFuPAS*. At the end of the paper, IDSs and adversarial models are repeatedly trained to achieve a dynamic balance between attack and defense. However, they do not consider the attack cost, which means that adversarial examples are restricted.

In general, adversarial attacks achieve excellent attack effects, but they lack enough restriction on adversarial attack models, which can cause invalid. In addition, the black-box scene is more generic than the white-box. Therefore, we propose a restricted adversarial attack model in the black-box scene.

3 Scene and Motivation

3.1 The Adversarial Attack Scene

In this paper, we focus on generating adversarial examples to escape IDSs. We attack ML-based IDSs in the black-box scene by adding perturbations to traffic features.

Targeting ML-Based IDSs: IDSs do not detect the actual traffic because the traffic data is very complex with unstructured data. Usually, ML-based IDSs

extract features of traffic to classify. The target of attacks is ML models. Specially, we will visually describe in Sect. 3.2 how attackers exploit flaws of ML models to attack.

Perturbations in Traffic Features: In the network traffic field, the adversarial attack is a difficult task. Different from other fields such as computer vision, images are continuously differentiable data and can be tampered with arbitrarily. However, traffic data is a discrete byte stream and is not differentiable. It is not feasible to directly use a gradient-based attack method on network traffic. Therefore, we attack based on the features of traffic. More precisely, we add perturbations to the features extracted from malicious traffic. If the adversarial features can attack ML models successfully, traffic with these features can also escape IDSs.

In Black-Box Scene: As mentioned in Sect. 2, we conduct adversarial attacks in the black-box scene. It is plug-and-play to attack any ML-based IDS.

3.2 Visual Description

We explain this process of attacks for a more visual description in Fig. 1. As shown in Fig. 1(a), triangles represent malicious examples, crosses represent benign examples, and two types of examples are separated by the line, which represents the decision boundary of a ML-based IDS. Attackers can add perturbations to malicious examples to escape the IDS.

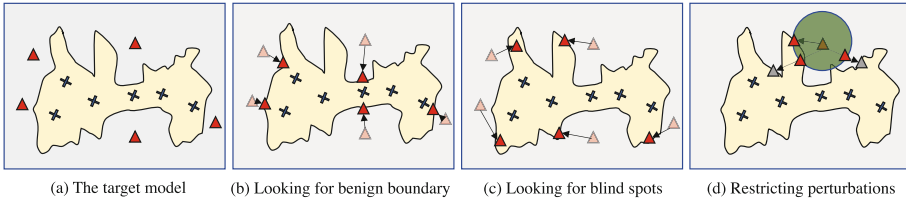


Fig. 1. Adversarial attack scene.

Looking for Benign Boundary: Obviously, mapping malicious examples completely into benign feature space is definitely able to attack any model. However, malicious and benign examples are fundamentally different, so they cannot have exactly the same features. As shown in Fig. 1(b), attackers can make it imitate benign examples and find the boundary of the ML-based IDS.

Looking for Blind Spots: In general, ML-based IDSs have weak robustness. Specifically, only part of high probability regions of examples are sufficiently trained, but low probability regions of examples are ignored. We call low probability regions blind spots of ML models, as shown in Fig. 1(c). Attackers can generate adversarial examples by exploring these blind spots of the model.

Restricting Perturbations: Attacks can not be unrestricted to avoid destroying the characteristic of malicious examples. Therefore, the perturbations of adversarial examples should be as small as possible, as shown in Fig. 1(d). It is necessary to keep adversarial examples within a small range.

In summary, attackers look for the boundary and blind spots of ML-based IDSs on a small scale, and generate adversarial examples to escape detection.

3.3 Motivation

In the previous related work, adversarial examples can usually achieve a high *EIR*. However, most adversarial examples lack sufficient restrictions. In particular, we summarize three following important characteristics of adversarial examples.

Regularization: Regularization means that adversarial examples should not violate rules of network protocols. For example, the length of traffic packets cannot exceed the maximum range specified by the protocol specification, and the protocol type of traffic packets is unchangeable.

Maliciousness: Maliciousness means that adversarial examples cannot destroy their functional features. For instance, for DoS attack traffic, the number of downstream bytes (from destination to source) should be much larger than the number of upstream bytes. If the preceding features are not met, DoS attack traffic loses maliciousness. Therefore, these functional features should remain the same.

Small Perturbation: Small perturbation means that adversarial examples should be sufficiently similar compared with original malicious examples. Features can not be modified without restriction. For example, attackers modify the packet sending frequency and packet lengths to attack. However, excessive perturbation can result in traffic packets being invalid.

In light of this, we restrict adversarial attacks so that adversarial examples not only have a high *EIR*, but also keep regularization, maliciousness and small perturbation.

4 Methodology

In this section, we describe the adversarial attacks with formula. Then the restricted adversarial attack model *RAAM* is introduced.

4.1 Math Description

As we describe in Sect. 3.3, attackers should keep adversarial examples regularization, maliciousness and small perturbation. Specifically, the ML model is defined as f , the input as x , and the output as $f(x)$. As shown in Eq. 1, for an adversarial attack, results of the model $f(x)$ should be misled by adding

perturbations δ as small as possible. R represents the restriction for adversarial examples to ensure matching actual traffic features.

$$\operatorname{argmin}_{\delta} f(R(x + \delta)) \neq f(x) \quad (1)$$

4.2 Model Architecture

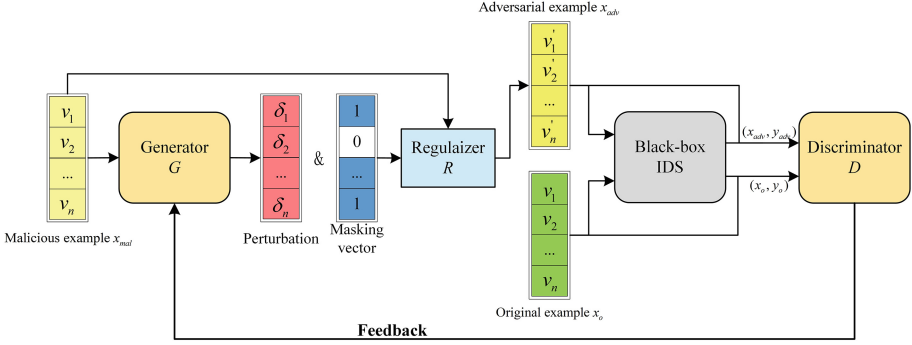


Fig. 2. The architecture of *RAAM* based on WGAN.

Overview. We briefly introduce our restricted adversarial attack model, *RAAM*, which is based on WGAN to solve the problem of unstable GAN training. As shown in Fig. 2, *RAAM* is mainly consisted of three components: a generator G , a discriminator D and a black-box IDS. First, the input of G is the original malicious example x_{mal} which is consisted of feature vector v_i , and the output is the perturbation δ . Then, δ is added to x_{mal} for generating the adversarial example x_{adv} . Specially, we come up with the masking vector and regularizer R to restrict δ and keep regularization, maliciousness and small perturbation. What's more, x_{adv} and the original example x_o including the original benign example and the original malicious example is input into the black-box IDS to get the label y . Finally, the example and the label are input to D for training. D imitates the detection ability of the IDS, while games with G through feedback.

Generator G . The generator G is a neural network for generation. It generates the perturbation δ with the same dimensions of the original example. However, as we discussed in Sect. 3.3, not all features of the example can be changed, and functional features should keep the same to maintain maliciousness. Therefore, the perturbation δ is multiplied by the masking vector composed of 0 and 1 as shown in Eq. 2, so that perturbation vectors δ_i of functional features are zeros.

$$\delta'_i = \begin{cases} 0, & \delta_i \in \text{functional features}, \\ \delta_i, & \delta_i \notin \text{functional features}. \end{cases} \quad (2)$$

Regularizer R . The regularizer R is to keep x_{adv} regularization and small perturbation. As shown in Fig. 1(d), perturbations δ should be in a restricted range. Specifically, a perturbation threshold ε is set to control the perturbation distance, as shown in Eq. 3. In addition, any feature vector of adversarial examples v_i can not exceed the range of actual traffic features as shown in Eq. 4.

$$\delta' = \begin{cases} \delta, & |\delta| \leq \varepsilon, \\ \frac{\varepsilon}{|\delta|} * \delta, & |\delta| > \varepsilon. \end{cases} \quad (3)$$

$$(\delta_i + v_i) \in [Min(v_i), Max(v_i)] \quad (i = 1 \sim n) \quad (4)$$

Black-box IDS. The black-box IDS is the target model of adversarial attack, which can be any trained ML model. The structure and parameters of the model do not need to be known, and only the output result is needed. Its input is adversarial examples x_{adv} and original examples x_o , and the output is y_{mal} and y_{ben} . The output is used as labels for motivational D training.

Discriminator D . The discriminator D is used to identify whether the example is malicious or benign. Different from the traditional WGAN model, it mainly includes three types of inputs, which are benign, malicious, and adversarial examples. D get their labels from the black-box IDS to train for distilling learning [23]. The output of D is a one-dimensional score, which is low for malicious and high for benign examples. Then it is feed back to G and D to train accordingly. G is motivated to generate highly adversarial examples to confuse D .

Improved Loss. WGAN mainly relies on the loss L_G and L_D of the generator G and the discriminator D to carry out iterative training. In L_G back propagation, parameters of D are frozen, while in L_D back propagation, parameters of G are frozen. G is trained by updating in weight corresponding to gradient information from scores of D . As shown in Eq. 5, scores of D are the L_G , which represents the difference between adversarial examples with original benign examples.

$$L_G = - \mathbb{E}_{\tilde{x} \in x_{adv}} [D(\tilde{x})] \quad (5)$$

For the discriminator D , on the one hand, L_D is calculated by the evaluation scores of D for adversarial examples x_{adv} and original benign examples x_o . On the other hand, D imitates detection capabilities of the black-box IDS to classify between original malicious examples x_{mal} and original benign example x_{ben} . Therefore, as shown in Eq. 6, we propose the improved loss so that D not only generates adversarial examples with G game, but also imitates the IDS better.

$$L_D = \underbrace{\mathbb{E}_{\tilde{x} \in x_{adv}} [D(\tilde{x})] - \mathbb{E}_{x \in x_{ben}} [D(x)]}_{\text{Original loss}} + \underbrace{\mathbb{E}_{\tilde{x} \in x_{mal}} [D(\tilde{x})]}_{\text{Improved loss}} \quad (6)$$

In general, the generator G and the discriminator D are balanced in the game. G tries to confuse D which has a similar detection ability to the black-box IDS by distilling learning. This indirectly implements adversarial attacks against the black-box IDS.

5 Experiment and Evaluation

In this section, we introduce the preparation of the adversarial attack experiment, including the data, training of black-box IDSs, and the implementation of *RAAM*. Then we define evaluation metrics and verify the performance of *RAAM* for generating adversarial examples against IDSs. Finally, we retrain IDSs by adversarial examples to resist adversarial attacks.

5.1 Dataset

Dataset Selection. We select the well-known NSL-KDD dataset to train and evaluate the performance of *RAAM*. NSL-KDD is an improved version of KDD-99 and has become the benchmark dataset of IDSs based on ML algorithms. As shown in Table 1, there are two separate subsets of NSL-KDD that can be used to train and test ML models, called KDDTrain+ and KDDTest+, respectively. In addition, NSL-KDD consists of five traffic categories, including Probe, DoS, U2R, R2L and benign traffic.

Table 1. The data sampling result of NSL-KDD.

Dataset	Total	Benign	DoS	Probe	U2R	R2L
KDDTrain+	125973	67343	45927	11656	52	995
KDDTest+	22544	9711	7458	2421	200	2654

Features Selection. NSL-KDD is extracted from real network traffic and has different categories of features. From the perspective of attributes, features can be divided into four categories: Intrinsic, Content, Time-based traffic and Host-based traffic [22]. As presented in Sect. 3.3, each category of attack traffic has unchangeable functional features, as shown in Table 2. *RAAM* can only change the nonfunctional features in order to avoid destroying the maliciousness.

Dataset Preprocessing. SMOTE sampling method is used to enhance the data and solve the problem of data imbalance. We also normalize data with Max-Min normalization. In addition, the training sets are divided into two parts: 50% is used for the training of IDSs, and the other is used for the training of *RAAM*.

Table 2. Functional features of attack categories in NSL-KDD dataset.

Attack	Intrinsic	Content	Time-based traffic	Host-based traffic
Probe	✓		✓	✓
DoS	✓		✓	
U2R	✓	✓		
R2L	✓	✓		

5.2 Evaluation Methods and Experimental Configurations

Evaluation Metrics. For the evaluation metrics, we use the method proposed by *IDSGAN*, which measures the detection rate (DR) and the evasion increase rate (EIR). As shown in Eq. 7, DR refers to the proportion of malicious traffic records $N_{detected}$ correctly detected by black-box IDSs to all attack records N , which directly reflects the escape ability of examples and the robustness of black-box IDSs. As shown in Eq. 8, EIR is the rate of the increase in the detection rate DR_{adv} for adversarial examples compared with the detection DR_o for original malicious examples to measure the evasive attack of *RAAM*.

$$DR = \frac{N_{detected}}{N} \quad (7)$$

$$EIR = 1 - \frac{DR_{adv}}{DR_o} \quad (8)$$

A lower DR represents that more malicious or adversarial examples can escape black-box IDSs. Conversely, a higher EIR represents that more adversarial examples can escape black-box IDSs than original malicious examples. Thus, the motivation for *RAAM* is to achieve a lower DR_{adv} and a higher EIR . In addition, we set $F1$ score to measure the ability of black-box IDSs to detect malicious and benign traffic. What’s more, the restriction of adversarial examples should also be considered besides adversarial performance. We represent the distance of perturbations by measuring the Euclidean distance (L_2) between the adversarial examples x_{adv} and the original malicious examples x_{mal} in Eq. 9.

$$L_2 = \sqrt{\sum_{i=1}^n |x_{adv}^i - x_{mal}^i|^2} \quad (9)$$

Hyper-parameters. Based on experience and previous experiments, we determined some basic parameters of the model. For WGAN, the epoch size is 50, the batch size is 128, and the learning rate of both discriminator and generator is 0.0001. The perturbation threshold ε is 4, and the cutting weight threshold of discriminator training is 0.01. Like the most basic GANs, the generator and the discriminator of *RAAM* are both multi-layer neural networks.

Experiment Setting. *RAAM* is implemented in Python3.7 based on Scikit-learn and Pytorch. The system is Ubuntu 16.04. All software applications are deployed on a server machine with 64 cores and 128GB memory. To further speed up matrix calculations, three NVIDIA TITAN XPs are deployed.

In order to detect the adversarial performance of *RAAM* in a more comprehensive way, nine current mainstream ML models are selected as black-box IDSs, including Logistic Regression (LR), Support Vector Machine (SVM), Naive Bayes (NB), Decision Tree (DT), Random Forest (RF), K-NearestNeighbor (KNN), Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), Recurrent Neural Network (RNN) models. LR, SVM, NB, DT, RF, KNN and MLP are based on Scikit-learn, while CNN, RNN and WGAN are based on PyTorch.

5.3 Escape Detection

We present our experimental results and evaluate the performance of *RAAM*. There are two external contrast experiments and one self-contrast experiment. The baseline experiment is that adversarial examples are added with blind random perturbations, called *Random* attack. The next contrast experiment is an advanced adversarial attack architecture, called *DIGFuPAS* [6]. In addition, the improved loss is used or removed to verify its effect. We select the same attack types to compare with *DIGFuPAS* expediently, including DoS, U2R and R2L attacks, respectively. In particular, U2R and R2L attacks are classified into one type U2R & R2L attack because they are similar and in small quantities. Table 3 shows the detection ability of different ML-based IDSs for original examples. Table 4 shows the results of adversarial attacks in the DoS attack, and the results of the U2R & R2L attack are shown in Table 5. Finally, we detect different adversarial performances by changing the perturbation threshold ε in Fig. 3.

Table 3. The detection results of original examples in NSL-KDD.

Attack	Metric	LR	SVM	NB	DT	RF	KNN	MLP	CNN	RNN	Mean
DoS	<i>F1</i> (%)	86.58	89.62	86.50	88.55	90.90	93.04	91.16	93.40	90.05	89.97
	<i>DR</i> (%)	85.03	82.99	86.70	94.13	95.09	94.90	95.40	98.39	98.14	92.30
U2R & R2L	<i>F1</i> (%)	80.71	71.60	80.17	77.41	78.18	80.41	79.27	78.76	82.28	78.75
	<i>DR</i> (%)	59.50	44.28	69.91	57.69	64.47	69.16	65.62	56.21	66.31	61.46

Original Detection. First, we evaluate the detection ability of the nine ML-based IDSs. As shown in the Table 3, in the DoS attack, the *F1* score of test sets can reach 86%–93%, and the detection rate *DR* can reach 82%–98% on various models. In the U2R & R2L attack, the *F1* score achieves 71%–82%, and the detection rate *DR* achieves 44%–69% because the number of the U2R & R2L attack is low. On the whole, ML-based IDSs can well identify malicious traffic and benign traffic.

Table 4. Compared performance of adversarial methods in the DoS attack.

Method	Metric	LR	SVM	NB	DT	RF	KNN	MLP	CNN	RNN	Mean
<i>Random</i>	<i>DR</i> (%)	86.07	87.81	13.49	30.36	30.20	80.56	84.36	75.52	90.34	64.30
	<i>EIR</i> (%)	-1.22	-5.80	84.44	67.74	68.24	15.11	11.57	23.24	7.94	30.14
	L_2	2.37	2.37	2.38	2.36	2.38	2.38	2.37	2.38	2.37	2.37
<i>DIGFuPAS</i>	<i>DR</i> (%)	0	0.61	0	18.35	9.54	4.94	1.14	0.55	74.56	12.18
	<i>EIR</i> (%)	100	99.2	100	77.50	88.02	93.81	98.62	99.33	9.52	85.11
	L_2	2.71	2.56	2.05	2.67	2.60	3.33	3.35	3.06	1.74	2.67
<i>RAAM</i> (Base)	<i>DR</i> (%)	30.16	0	68.45	9.78	18.86	27.02	14.32	0.37	18.06	20.78
	<i>EIR</i> (%)	64.53	100	21.04	89.61	80.16	71.52	84.98	99.62	81.59	77.00
	L_2	1.48	2.05	1.93	2.06	2.25	2.20	1.51	1.76	1.91	1.90
<i>RAAM</i> (Improved)	<i>DR</i> (%)	11.07	0.07	0.64	9.48	9.47	0.75	0.06	0.12	7.11	4.30
	<i>EIR</i> (%)	86.98	99.91	99.26	89.92	90.03	99.20	99.94	99.88	92.75	95.31
	L_2	1.99	1.83	2.03	1.43	2.20	2.05	2.29	1.93	2.05	1.97

Comparison of Adversarial Effects. As shown in Table 4, all attack methods except *Random* can interfere with the detection of IDSs in the *DoS* attack. This shows that ML-based IDSs are easily attacked by adversarial attacks. In various IDSs, the mean *DR* of *RAAM* is 4.30% which is 60% lower than *Random* and 7.88% lower than *DIGFuPAS*, and the mean *EIR* is 95.31% which is 64.91% higher than *Random* and 10.2% higher than *DIGFuPAS*. Compared with the other two methods, *RAAM* has stronger adversarial effects. We think this is mainly because *RAAM* uses the improved loss so that the discriminator *D* can better imitate black-box IDSs. We will prove its effect through the comparison experiment in later. In addition, the mean distance of perturbations L_2 of *RAAM* is 1.97, which is 0.4 lower than *Random* and 0.7 lower than *DIGFuPAS*. This shows that the restriction of *RAAM* is effective and adversarial examples have lower perturbations.

For the U2R & R2L attack, *DIGFuPAS* does not use SMOTE sampling in the original paper, so its black-box IDSs are poor in detecting these types of attacks with a small number of examples. Therefore, we use the results reproduced by ourselves instead of the results in their paper. As shown in Table 5, the mean *DR* and *EIR* of *RAAM* are 4.76% and 92.88% respectively, which are also lower and higher than the other two methods. For the perturbations, the mean L_2 of *RAAM* is 1.61, which is also the lowest.

It is worth mentioning that we find that the RF model and the DT model are the least affected by adversarial attacks. What they have in common is that they are all tree models. We think this is because the tree models are least sensitive to irrelevant data, and some researchers have also proved that tree models have stronger stability [24].

Improved Loss. We research the impact of improved loss by removing it. As shown in Table 4 and Table 5, when improved loss is removed, *EIR* decreased significantly in multiple black-box IDSs such as SVM, NB, KNN, *etc.* We believe that this is because the discriminator *D* with the base loss cannot transfer the ability to learn black-box IDSs, so that the generator *G* has no motivation to generate more deceptive adversarial examples for black-box IDSs.

Table 5. Compared performance of adversarial methods in the U2R & R2L attack.

Method	Metric	LR	SVM	NB	DT	RF	KNN	MLP	CNN	RNN	Mean
<i>Random</i>	<i>DR</i> (%)	58.53	49.62	92.68	35.53	25.27	57.48	51.99	73.71	70.83	57.29
	<i>EIR</i> (%)	1.63	-12.0	80.66	-32.5	60.8	16.87	20.75	-31.1	-6.80	10.92
	L_2	2.49	2.49	2.50	2.49	2.48	2.49	2.49	2.50	2.48	2.49
<i>DIGFuPAS</i>	<i>DR</i> (%)	0	0	0.74	15.45	14.10	25.21	0.48	0	19.27	8.36
	<i>EIR</i> (%)	100	100	98.94	73.21	78.12	63.56	99.26	100	70.93	87.11
	L_2	2.34	2.73	2.46	2.90	2.33	2.07	2.33	3.06	2.42	2.51
<i>RAAM</i> (Base)	<i>DR</i> (%)	1.05	0	17.58	63.88	0.13	55.38	0.27	99.42	0.47	26.46
	<i>EIR</i> (%)	98.23	100	74.85	-10.7	99.78	19.92	99.58	-76.8	99.28	56.01
	L_2	1.27	2.50	1.81	1.36	1.83	2.11	1.89	1.36	1.27	1.71
<i>RAAM</i> (Improved)	<i>DR</i> (%)	1.65	0	17.58	6.23	0.23	16.86	0	0	0.33	4.76
	<i>EIR</i> (%)	97.21	100	74.85	89.19	99.63	75.60	100	100	99.49	92.88
	L_2	0.89	1.56	1.18	1.31	1.73	2.29	1.74	2.25	1.51	1.60

Effect of Different Perturbations. We observe adversarial performance by changing the perturbation threshold ε . In this way, the robustness of different ML models are evaluated in Fig. 3 shows experimental results with different threshold ε in the DoS attack NSL-KDD. Even if perturbation threshold ε reaches an extremely low value of 1, *DR* can remain 30%–50% in most black-box IDSs. In addition, KNN and LR models perform well under sufficiently small perturbations, while the RNN and RF models perform weakly. In general, the more complex the model can be attacked by the lower perturbations.

5.4 Detection Effect of Enhanced IDSs

In this subsection, we research how IDSs defend against attackers through adversarial training. Specifically, we use generated adversarial examples to retrain IDSs for enhancement. Then we attack the retrained IDSs again to detect their robustness. Note that the previous adversarial examples are not used, but the new generation for attacking the retrained IDSs again in the black-box scene.

In the experiment, we use adversarial examples generated by *RAAM* to retrain, and use different adversarial methods to attack the enhanced IDSs. We mainly consider two points of the enhanced IDSs, (1) it has stronger robustness for adversarial examples; (2) it does not interfere with the detection of original examples. Therefore, we observe two main metrics, *DR* of adversarial examples and *F1* score of original examples.

We conduct experiments in the DoS attack as an illustration. As shown in Table 6, the first two lines represent the comparison results for the detection of the original examples, and the other lines represent the detection of the three attacks. After retraining, *F1* score does not decrease significantly and even rises slightly in several ML-based IDSs. This shows that enhanced IDSs still have a good ability to detect original examples. *DR* increases by 36.89% on average against *DIGFuPAS*, and *DR* increases by 25.72% on average against *RAAM*. In general, the enhanced IDSs are more robust for adversarial attacks without affecting the detection of original examples.

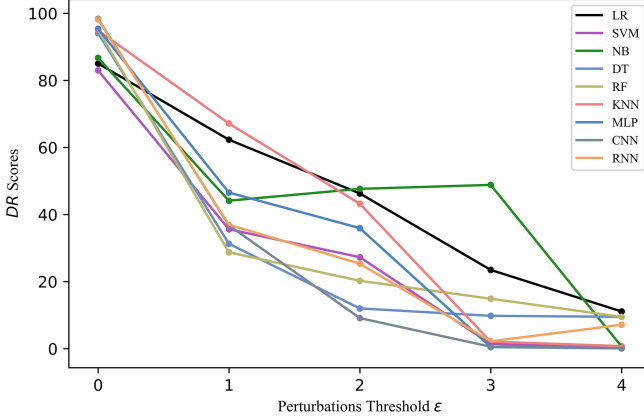


Fig. 3. DR of different perturbation threshold ϵ in the DoS attack.

Table 6. The comparative results of previous and retrained IDSs in the DoS attack of NSL-KDD.

Method	Metric	LR	SVM	NB	DT	RF	KNN	MLP	CNN	RNN	Mean
Original	<i>F1 (%)</i>	86.58	89.62	86.50	88.55	90.90	93.04	91.16	93.40	90.05	89.97
+Retrained	<i>F1 (%)</i>	85.01	86.67	82.57	88.42	91.03	91.80	92.36	90.93	89.83	88.73
<i>Random</i>	<i>DR (%)</i>	86.07	87.81	13.49	30.36	30.20	80.56	84.36	75.52	90.34	64.30
+Retrained	<i>DR (%)</i>	86.54	87.83	25.60	39.85	28.71	81.47	82.77	88.54	91.05	68.04
<i>DIGFuPAS</i>	<i>DR (%)</i>	0	0.61	0	18.35	9.54	4.94	1.14	0.55	74.56	12.18
+Retrained	<i>DR (%)</i>	85.22	56.01	0.10	9.67	20.21	31.27	72.05	94.50	72.74	49.08
<i>RAAM</i>	<i>DR (%)</i>	11.07	0.07	0.64	9.48	9.47	0.75	0.06	0.12	7.11	4.30
+Retrained	<i>DR (%)</i>	58.79	13.85	57.20	19.09	19.10	35.15	22.94	25.92	18.24	30.03

6 Discussion

In this paper, we propose a restricted adversarial attack with adding smaller perturbations distance. We think that the fewer number of perturbation features is also important for restricted adversarial attack. We intend to modify only one or several nonfunctional features for attacks. This can better preserve the nature of the original malicious traffic.

In addition, our work only conducts adversarial attacks against binary-classification models. In other fields, there are some adversarial attacks against multi-classification models, call targeted attacks. Attackers can misidentify a example from one category to a particular category. In the network traffic detection field, multi-classification tasks are increasingly common. Therefore, it is meaningful to conduct targeted attacks against multi-classification models.

7 Conclusion

In this paper, we focus on issues that adversarial attacks should have more sufficient restrictions. We propose *RAAM*, a restricted adversarial attack model. We attack different ML-based IDSs in the black-box scene, and test with the NSL-KDD dataset. WGAN with the improved loss is used for imitating IDSs, and gets a higher mean *EIR* 94.1%. Regularizer *R* and the masking vector are used to restrict attacks, and get a lower mean distance of perturbations L_2 1.79. In addition, we retrain IDSs to improve their robustness. Experimental results show that retrained IDSs can weaken most adversarial attacks without affecting the detection of original examples.

In the future, we intend to conduct adversarial attacks with fewer features for multi-classification, and target more advanced and complex ML-based IDSs.

Acknowledgement. This work is supported by the National Key Research and Development Program of China (Grant No.2018YFB0804704).

References

1. Szegedy, C., et al.: Intriguing properties of neural networks. arXiv preprint [arXiv:1312.6199](https://arxiv.org/abs/1312.6199) (2013)
2. Aiken, J., Scott-Hayward, S.: Investigating adversarial attacks against network intrusion detection systems in SDNs. In: IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), pp. 1–7 (2019)
3. Huang, C.-H., Lee, T.-H., Chang, L., Lin, J.-R., Horng, G.: Adversarial attacks on SDN-Based deep learning IDS system. In: Kim, K.J., Kim, H. (eds.) ICMWT 2018. LNEE, vol. 513, pp. 181–191. Springer, Singapore (2019). https://doi.org/10.1007/978-981-13-1059-1_17
4. Lin, Z., Shi, Y., Xue, Z.: IDSGAN: generative adversarial networks for attack generation against intrusion detection. arXiv preprint [arXiv:1809.02077](https://arxiv.org/abs/1809.02077) (2018)
5. Usama, M., Asim, M., Latif, S., et al.: Generative adversarial networks for launching and thwarting adversarial attacks on network intrusion detection systems. In: 2019 15th international Wireless Communications & Mobile Computing Conference (IWCMC), pp. 78–83 (2019)
6. Duy, P.T., Khoa, N.H., Nguyen, A.G.T., et al.: DIGFuPAS: deceive IDS with GAN and function-preserving on adversarial examples in SDN-enabled networks. *Comput. Secur.* **109**, 102367 (2021)
7. Han, D., Wang, Z., Zhong, Y., et al.: Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors. *IEEE J. Sel. Areas Commun.* **39**, 2632–2647 (2021)
8. Hashemi, M.J., Cusack, G., Keller, E.: Towards evaluation of NIDSs in adversarial setting. In: Proceedings of the 3rd ACM CoNEXT Workshop on Big Data, Machine Learning and Artificial Intelligence for Data Communication Networks, pp. 14–21 (2019)
9. Xie, J., Li, S., Yun, X., et al.: HSTF-model: an HTTP-based Trojan detection model via the hierarchical spatio-temporal features of traffics. *Comput. Secur.* **96**, 101923 (2020)

10. van Ede, T., Bortolameotti, R., Continella, A., et al.: FlowPrint: semi-supervised mobile-app fingerprinting on encrypted network traffic. In: Network and Distributed System Security Symposium (NDSS), vol. 27 (2020)
11. Han, D., Wang, Z., Chen, W., et al.: DeepAID: interpreting and improving deep learning-based anomaly detection in security applications. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, pp. 3197–3217 (2021)
12. Xie, G., Li, Q., Jiang, Y.: Self-attentive deep learning method for online traffic classification and its interpretability. *Comput. Netw.* **196**, 108267 (2021)
13. Hu, W., Tan, Y.: Generating adversarial malware examples for black-box attacks based on GAN. arXiv preprint [arXiv:1702.05983](https://arxiv.org/abs/1702.05983) (2017)
14. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. arXiv preprint [arXiv:1412.6572](https://arxiv.org/abs/1412.6572) (2014)
15. Papernot, N., McDaniel, P., Jha, S., et al.: The limitations of deep learning in adversarial settings. In: IEEE European Symposium on Security and Privacy (EuroS&P), pp. 372–387 (2016)
16. Liu, C., Ding, W., Hu, Y., et al.: Rectified binary convolutional networks with generative adversarial learning. *Int. J. Comput. Vis.* **129**(4), 998–1012 (2021)
17. Demetrio, L., Biggio, B., Lagorio, G., et al.: Functionality-preserving black-box optimization of adversarial windows malware. In: IEEE Transactions on Information Forensics and Security, pp. 3469–3478 (2021)
18. Rahman, M.S., Imani, M., Mathews, N., et al.: Mockingbird: defending against deep-learning-based website fingerprinting attacks with adversarial traces. *IEEE Trans. Inf. Forensics Secur.* **16**, 1594–1609 (2020)
19. Gulrajani, I., Ahmed, F., Arjovsky, M., et al.: Improved training of Wasserstein GANs. arXiv preprint [arXiv:1704.00028](https://arxiv.org/abs/1704.00028) (2017)
20. Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al.: Generative adversarial nets. *Adv. Neural Inf. Process. Syst.* **27** (2014)
21. Tavallaei, M., Bagheri, E., Lu, W., Ghorbani, A.: A detailed analysis of the KDD CUP 99 data set. In: Submitted to Second IEEE Symposium on Computational Intelligence for Security and Defense Applications (2009)
22. Davis, J.J., Clark, A.J., et al.: Data preprocessing for anomaly based network intrusion detection: a review. *Comput. Secur.* **30**, 353–375 (2011)
23. Hinton, G., Vinyals, O., Dean, J.: Distilling the Knowledge in a neural network. *Stat* 1050–1058 (2015)
24. Shwartz-Ziv, R., Armon, A.: Tabular data: deep learning is not all you need. arXiv preprint [arXiv:2106.03253](https://arxiv.org/abs/2106.03253) (2021)